

Maxey0 System One

A provider-neutral decision layer for Structured Context Worlds — semantic decision, context admission, authority boundaries, and observable agent execution

By Mike Cohan and Maxey0-ChatGPT 5.5 · Maxey0 — AI Systems Architecture and Research
human@maxey0.com · September 2026 · introductory research paper (architecture proposal)

Abstract

Autonomous AI systems are evolving from single model calls into persistent computational systems that retrieve information, maintain state, select tools, invoke external services, delegate tasks, coordinate multiple agents, modify files, and execute code. As these systems become more capable, the central architectural problem increasingly concerns not only what a model decides, but where that decision is permitted to operate, what contextual state is visible to it, which capabilities are addressable, which transitions are admissible, and how the resulting execution can be reconstructed. This paper presents Maxey0 System One as a research application of the Maxey0-SuperSpace architecture. The underlying architecture proposes Structured Context Worlds (SCWs) as explicit semantic and operational address spaces; System One is a provider-neutral semantic decision layer operating over those address spaces. It selects among bounded candidate destinations — capabilities, skills, context regions, agents, tools, resources, or execution paths — while deliberately remaining separate from authorization and execution.

The principal architectural invariant is $\text{Decision} \neq \text{Authorization} \neq \text{Execution} \neq \text{Observation}$. A System One decision does not itself grant permission to act: the host evaluates it against identity, policy, capabilities, SCW state, lifecycle, provenance, and execution conditions before admission, and execution occurs only after that independent step. The complete operational chain is $\text{Context} \rightarrow \text{Candidate Set} \rightarrow \text{Decision} \rightarrow \text{Admission} \rightarrow \text{Execution} \rightarrow \text{Observation}$. No empirical performance or safety result is claimed here. The paper defines the architecture, applications, hypotheses, threat model, evaluation methodology, and implementation program required to test whether explicit semantic address spaces measurably improve context isolation, routing attribution, capability control, provenance, replay, drift detection, cache validity, multi-agent coordination, and the safety-productivity tradeoff.

Keywords: agentic AI, autonomous agents, Structured Context World, SCW, semantic routing, context isolation, agent safety, MCP, A2A, authorization, provenance, observability, decision systems, agent orchestration, context engineering, decision caching, execution-space control

Contents

- 1 Introduction: from model calls to execution systems
- 2 Research position and motivation
- 3 Structured Context Worlds and the SuperSpace
- 4 System One: a provider-neutral decision layer
- 5 Applications of the decision layer
- 6 Provenance, observability, and replay
- 7 Security model, threat model, and invariants

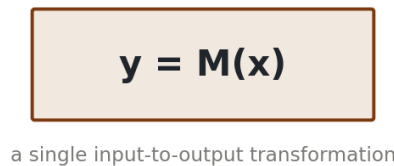
8	Relationship to existing agent infrastructure
9	The Maxey0 control architecture: SCW0 and the Dyson interpretation
10	Evaluation methodology
11	Implementation
12	Limitations and what System One does not claim
13	Discussion and conclusion
	Appendix A Core formal model · Appendix B Decision / admission / execution contracts · Note on sources · References

1 Introduction: from model calls to execution systems

The conventional abstraction for a language model is a transformation from an input to an output, $y = M(x)$. That abstraction remains useful, but it no longer describes what a modern autonomous system actually is.

An agent now receives a task, retrieves information, constructs context, selects a skill, invokes a tool, delegates to another agent, inspects files, executes code, receives a result, updates persistent state, and continues operating. The computational object is therefore no longer a model invocation. It is a stateful system of models, context, capabilities, environments, protocols, policies, and execution state. A more representative abstraction replaces the single arrow with a pipeline in which contextual state, an explicit decision, an independent admission result, execution, and a resulting state are each distinct.

The conventional abstraction



A more representative abstraction for an autonomous system

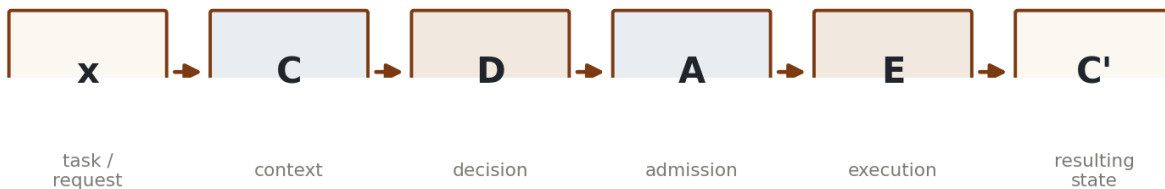


Figure 1. The abstraction has changed. A single input-to-output transform (top) understates what an autonomous system does; the operational reality (bottom) threads task, context, decision, admission, execution, and resulting state as separate objects.

Here x is the task or request, C is contextual state, D is a decision, A is an admission or authorization result, E is execution, and C' is the resulting state. This distinction becomes decisive the moment more than one agent operates concurrently. A multi-agent system may contain multiple contexts, multiple tool sets, multiple execution environments, multiple identities, multiple delegated tasks, and multiple forms of state. Contemporary multi-agent documentation already describes subagents that hold their own contexts under a coordinating parent, and notes that agents editing shared files must coordinate their changes.

1.1 Contextual independence is not execution isolation

The existence of independent model contexts does not, by itself, establish execution isolation. An agent may hold an independent model context while still sharing a filesystem, an MCP server, credentials, a network, authority inherited from a parent process, or durable state. The distinction between contextual independence and execution isolation is therefore fundamental, and it is precisely the distinction that implicit architectures blur. Two agents given “separate contexts” can still reach the same writable directory, and a directory reachable by both is a

communication channel whether or not anyone intended it to be.

Maxey0-SuperSpace addresses this by treating context as an addressable system resource rather than an undifferentiated pool of tokens. The foundational proposition is that context is not unbounded information but a structured object — an identity, a semantic scope, a state, provenance, policy, and a lifecycle. A Structured Context World is proposed as the bounded address space in which those elements, together with the capabilities and execution relationships that belong to a unit of work, are explicitly represented. System One is the decision layer built to operate over that structure.

2 Research position and motivation

This paper does not claim that semantic routing, authorization, sandboxing, agent orchestration, or observability are individually novel. Modern systems already provide many of these capabilities. Container and operating-system isolation can establish execution boundaries. Identity-and-access-management and capability systems can establish authorization boundaries. Vector databases and retrieval systems can select semantically related information. Agent frameworks can coordinate model calls and tools. The Model Context Protocol (MCP) can standardize access to tools and resources. Agent-to-agent messaging (A2A) can standardize communication among independent agents. Telemetry systems can record execution events, and policy engines can enforce rules.

The proposed research contribution is instead the composition of these concerns around an explicit semantic address-space abstraction. The central question is whether autonomous execution becomes more controllable when the system represents, as first-class objects: where an agent is operating; what context belongs to that location; what other locations are reachable; what capabilities exist at those locations; what semantic decision selected a destination; what independent authority admitted the transition; what runtime executed it; and what evidence records the resulting state transition. The hypothesis is architectural rather than dependent on any one model provider.

Motivation follows directly from the widening separation between model reasoning and system execution. A model may emit “use the financial-data tool.” The host must then determine whether that tool exists, whether it is available to this agent, whether the current principal is authorized to use it, whether the destination context permits access, whether the tool’s credentials are valid, and whether execution should occur at all. The model’s statement is not equivalent to authorization; semantic similarity is not equivalent to admissibility; a cached routing decision may remain plausible while being invalid under a newly changed policy. These motivate a chain of separations that the rest of the paper makes explicit.

Semantic suitability ≠ Context admissibility ≠ Authorization ≠ Execution

The current agent ecosystem already reflects part of this separation: managed agent harnesses distinguish a control plane — agent loops, model calls, tool routing, handoffs, approvals, tracing, recovery, and run state — from a sandbox execution plane in which files, commands, packages, ports, and state are manipulated. That precedent establishes that control and execution are already treated as different architectural responsibilities. System One proposes one further layer inside that separation: a semantic decision plane between structured context and host enforcement.

3 Structured Context Worlds and the SuperSpace

3.1 Context as an address space

A conventional context window is modeled as a sequence of information supplied to a model. SuperSpace instead treats context as an object with identity, scope, state, provenance, policy, lifecycle, relationships to other contexts, and execution associations. The proposed representation of a single world is a six-tuple.

$$\text{SCW}_i = (I_i, \Sigma_i, \Gamma_i, \Pi_i, L_i, \Omega_i)$$

Here I is the identity of the world, Σ its semantic scope, Γ its current contextual and operational state, Π its associated policy, L its lifecycle, and Ω its provenance and observability state. The notation is not the point; the point is that context becomes an addressable systems object rather than an implicit collection of prompt tokens. An SCW is therefore not merely a prompt container. It is an architectural boundary.

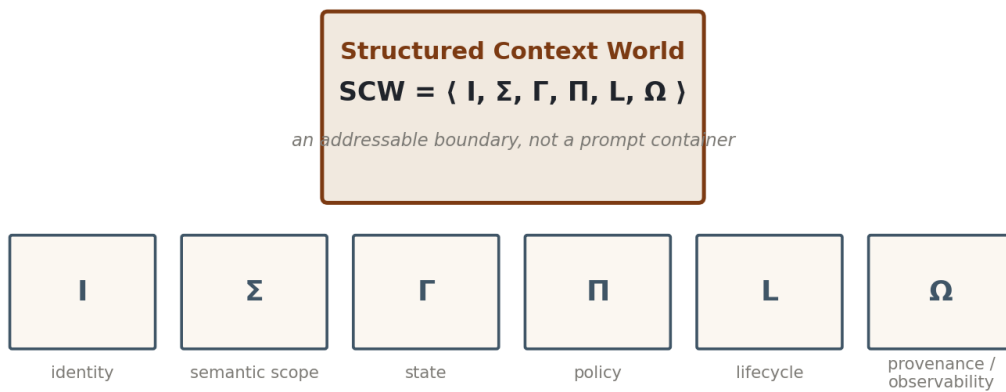


Figure 2. A Structured Context World as a six-tuple. Each component — identity, semantic scope, state, policy, lifecycle, and provenance/observability — is explicit and independently enforceable.

A Structured Context World can contain or reference model instructions, concepts, skills, agent and loop state, memory, retrieved documents, MCP resources and tools, A2A relationships, execution environments, policy metadata, provenance, decision records, artifacts, and durable state. The boundary determines which of these objects are visible or addressable. That produces two distinctions the rest of the architecture depends on.

Addressability ≠ Admissibility · Semantic similarity ≠ Authorization

An object may be addressable as a candidate without being authorized for use, and a destination that is semantically relevant is not necessarily one to which the current agent may transition. These are the same category error viewed from two angles, and collapsing either one is how an implicit architecture lets a model's judgment become a security decision.

Addressability $\neq$ Admissibility Semantic similarity $\neq$ Authorization

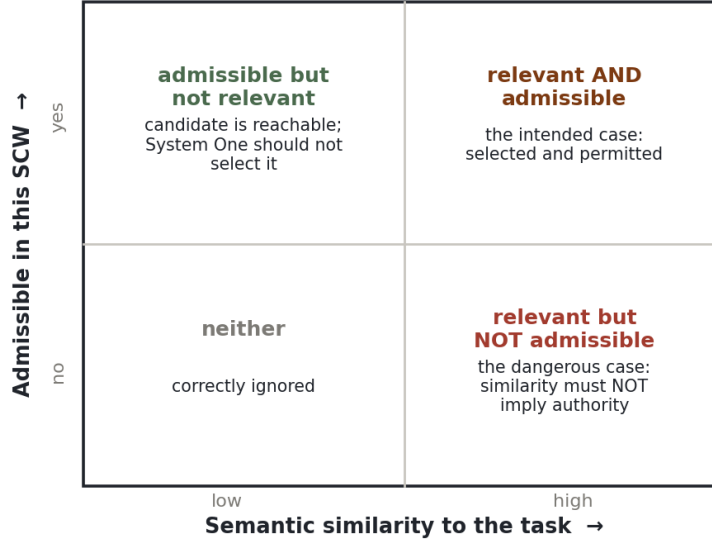


Figure 3. The two distinctions as a single concept map. The dangerous quadrant — relevant but not admissible — is exactly where similarity must not be allowed to imply authority. (Conceptual; no measured data.)

3.2 The SCW as an enforceable address space

The purpose of an SCW is not to replace operating-system isolation. Rather, it establishes a semantic and operational representation that can be mapped onto an underlying enforcement mechanism. An implementation can bind an SCW to a sandbox, so that the sandbox supplies the physical execution boundary while the SCW supplies the semantic boundary. The two remain distinct. A sandbox determines which files can be accessed, which processes can run, which network interfaces are available, and which operating-system permissions exist. An SCW additionally represents why the agent is operating there, which semantic task owns the context, which context is admissible, which other worlds are reachable, which capability selection led there, what provenance accompanies the transition, which decision produced the candidate, and which policy version governed admission.

$$\text{SCW} = \text{Semantic / Operational Address Space} \cdot \text{Physical Enforcement} = \text{Sandbox, OS, Network, IAM, Capability system}$$

The architecture is therefore explicitly not “SCW = Sandbox.” A sandbox can establish where code can physically execute; an SCW can establish why that execution belongs there. Binding a logical address to a physical enforcement location is the central engineering problem: an agent that discovers a physical path bypassing the logical abstraction has, in effect, found an unbound edge, and the value of the whole model depends on keeping the two in correspondence.

3.3 The SuperSpace graph and explicit bridges

A collection of SCWs forms a SuperSpace, represented as a graph $G = (V, E)$ whose vertices are worlds and whose edges are explicitly defined transitions, or bridges. A bridge is not a network connection; it is a semantic, policy-bearing transition, written as a tuple of source, destination, transfer mode, transferred representation, and governing policy.

$$B_{A \rightarrow B} = \langle A, B, m, r, p \rangle$$

The absence of a bridge means that state should not implicitly cross the boundary. System One can propose a bridge; it cannot create authority merely by proposing one. The resulting sequence is candidate bridge → admission → bridge execution, never candidate → immediate transfer. This is what lets a research platform measure whether explicit bridge representation reduces accidental context transfer: the destination receives only the permitted representation rather than inheriting the complete source context, which supports deliberate context minimization and makes every cross-context transfer an observable event.

SuperSpace $G = (V, E)$: worlds are nodes, bridges are explicit, policy-bearing edges

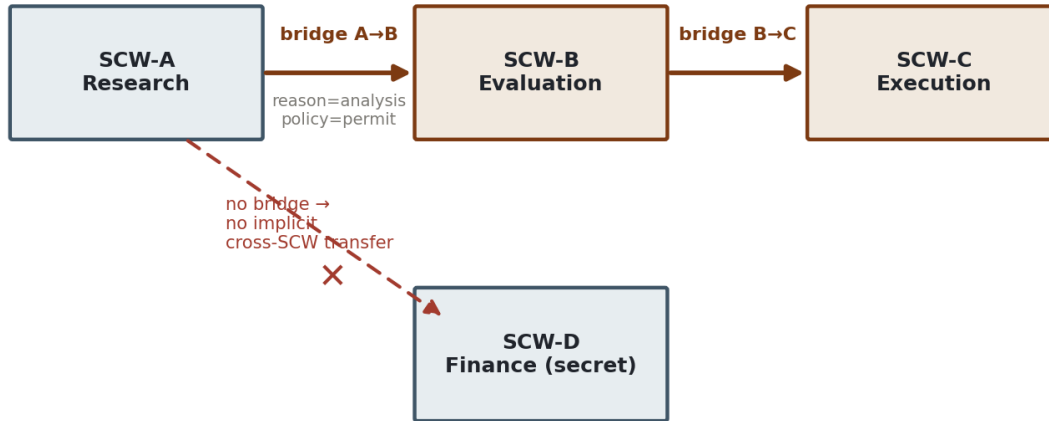


Figure 4. The SuperSpace graph. Authorized bridges are explicit and carry a reason and a policy; where no bridge exists, no implicit cross-SCW transfer is permitted, so a world holding secret state is not reachable merely because it exists.

3.4 The two-plane model

The Maxey0 architecture separates two principal planes. The Execution Plane contains the mechanisms that perform work — agents, loops, model invocations, tools, MCP, A2A, execution environments, subprocesses, coordination, delegation, retries, and external calls. It answers the question “how is work performed?” The Context Plane contains the semantic organization and boundaries that govern work — topics, concepts, skills, SCWs, context regions, semantic locations, prompts, resources, provenance, and context policy. It answers a different question: “where does the work belong?”

Keeping these planes distinct matters because they fail differently and are governed differently. The Execution Plane is where capability lives and therefore where damage happens; the Context Plane is where meaning and permission live and therefore where damage is authorized or prevented. Most implicit agent stacks fuse the two: the same reasoning step that decides what a task means also reaches for the tool that acts on it, so a mistake about meaning becomes a mistake about action with nothing in between. System One is positioned exactly at the boundary between the planes. It can evaluate which execution destination best corresponds to the current contextual state without becoming the mechanism that executes that destination.

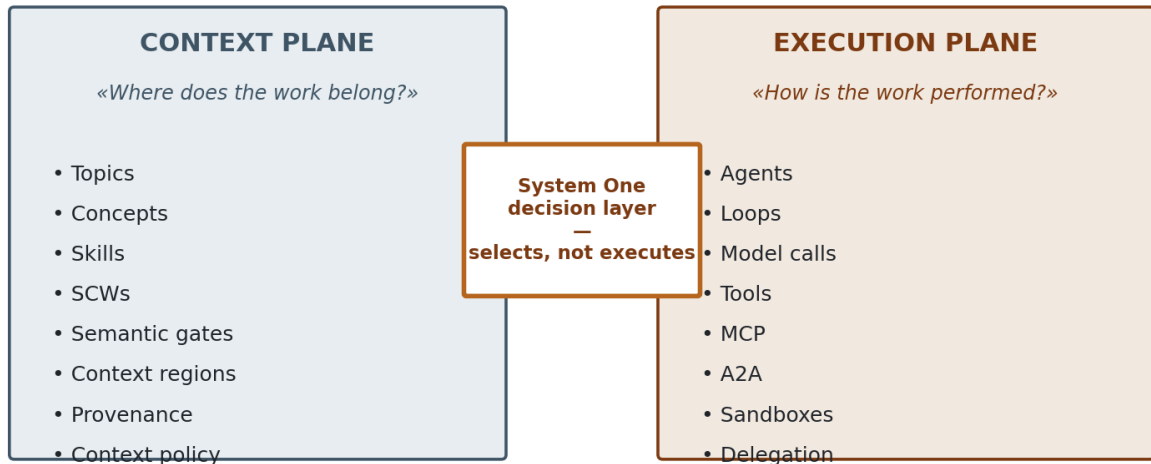


Figure 5. The two-plane model. The Context Plane answers “where does the work belong?” and the Execution Plane answers “how is it performed?” System One sits at the boundary, selecting without executing.

4 System One: a provider-neutral decision layer

System One is proposed as a provider-neutral semantic decision subsystem. It is not an agent harness, a sandbox, a language model, or an authorization system. It receives a bounded decision request, an explicitly defined contextual representation, a candidate set, relevant constraints, and execution state where appropriate; it produces a typed decision. Its purpose is to make the semantic decision explicit enough that the host can inspect, validate, authorize, execute, and record it.

$$\mathbf{d} = \mathbf{D}(\mathbf{x}, \mathbf{C}, \mathbf{S}, \mathbf{P}, \mathbf{G}, \mathbf{M}, \mathbf{V})$$

Here $\mathbf{x}$ is the decision request, $\mathbf{C}$ the admissible context, $\mathbf{S}$ execution state, $\mathbf{P}$ relevant policy constraints, $\mathbf{G}$ the candidate graph, $\mathbf{M}$ the decision model or provider, and $\mathbf{V}$ relevant version information. The output is a typed object, not free-form text, and it is emphatically not execution authority.

4.1 Provider neutrality

System One is deliberately decoupled from any single model provider. A provider adapter translates between a platform’s native decision interface and the normalized System One representation, so that a decision produced by a hosted model, a local model, or a deterministic test implementation is converted into the same native decision object, $D_{\text{native}} = N(D_{\text{provider}})$.

Provider substitution must not change SCW identity, SCW boundaries, admission semantics, capability enforcement, provenance requirements, execution semantics, or the observation schema. This is not a convenience; it is an experimental necessity. Without holding the execution architecture constant while varying the decision engine, any measured improvement could be an artifact of one model happening to route better rather than evidence for the architecture itself. A deterministic selector therefore serves as a control condition, and a provider adapter is never permitted to silently convert a decision into execution authority.

Provider substitution changes the decision engine — never the SCW, admission, or execution semantics

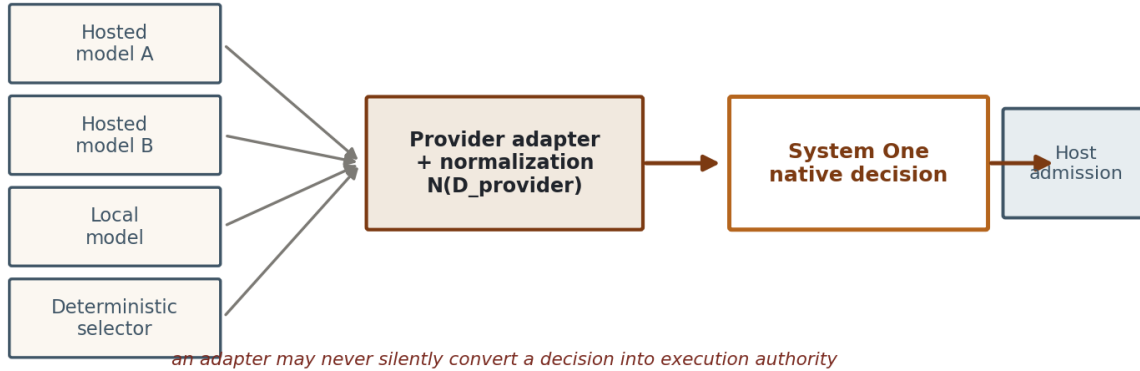


Figure 6. Provider neutrality. Any decision engine — hosted, local, or deterministic — is normalized to one native decision object; the SCW, admission, and execution semantics are held constant so the architecture’s contribution can be separated from the model’s.

4.2 The core invariants: decision, admission, execution, observation

The most important property of System One is that a decision is not an authorization. Suppose System One selects SCW_{17} as the most relevant destination for a task. That means only that SCW_{17} is the selected candidate under the supplied decision context. It does not mean the system is authorized to enter SCW_{17} . The host admission controller evaluates the transition separately, against identity, current context, current policy, capabilities, lifecycle, and provenance; only if it returns allow does execution become eligible.

A second invariant follows: an authorization result may permit a transition without executing it. The runtime remains responsible for spawning a process, invoking a tool, contacting an MCP server, sending an A2A message, mounting a resource, modifying a file, or calling an external service. This yields three distinct events — decision, admission, execution — each with its own identifier (`decision_id`, `admission_id`, `execution_id`). The separation is what makes incident analysis tractable: a system can determine whether the model selected the wrong destination, the policy admitted the wrong destination, the runtime executed something different from what was admitted, the environment changed after admission, or a tool behaved unexpectedly. Without these distinctions, every one of these failures appears simply as “the agent did something wrong.”

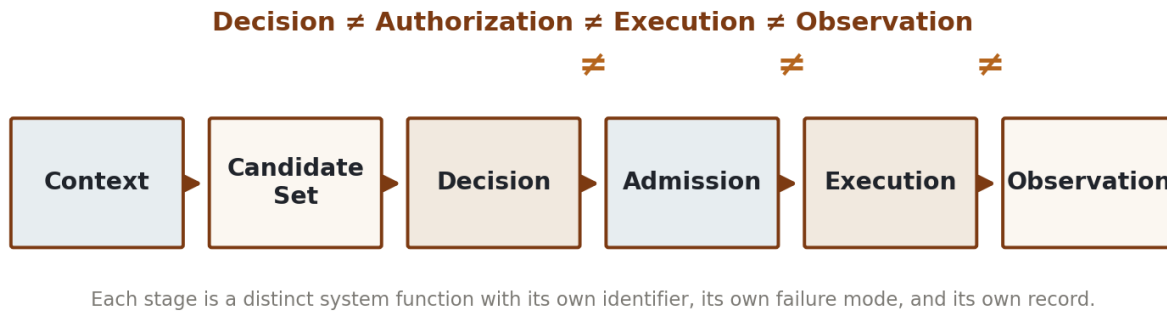


Figure 7. The complete operational chain and its separation invariant. Each stage is a distinct system function with its own identifier, its own failure mode, and its own record.

Extending the chain by one stage gives the four-stage model that organizes the rest of the paper: decision, admission, execution, and observation. Observation does not prevent an action; it makes the action reconstructable. The value of the four-stage view is that it partitions end-to-end failure into four independently attributable classes, which prevents a system failure from being misattributed to the model.

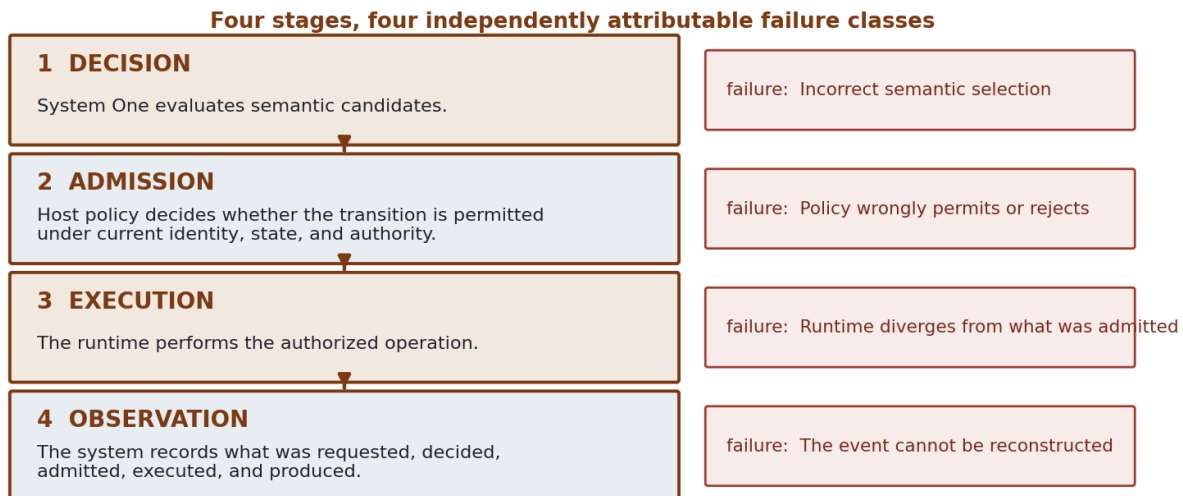


Figure 8. Four stages, four failure classes. Separating them keeps an execution or policy failure from being charged to the model's reasoning, and vice versa.

4.3 The decision object

A System One native decision is a typed record. Its authoritative fields identify the decision and the state that produced it; separate fields carry model-generated explanation and diagnostics. The distinction is deliberate: a model's natural-language rationale should never automatically be treated as authoritative evidence. The authoritative record is the structured decision together with the independently recorded inputs that produced it.

Identity	decision_id, request_id, run_id, task_id	correlate the decision across the run
Location	scw_id, source_location, selected_candidate, decision_type	what was selected, and where from
Inputs (digested)	candidate_set_digest, context_digest, state_digest	bind the decision to the exact state it saw
Provenance	provider, model, model_version, policy_version, timestamp, provenance	who/what/when, for replay and audit
Diagnostic (non-authoritative)	confidence, ranking, evidence, reason_code, alternative_candidates, semantic_distance, cache_status	explanation and analysis only

Because the inputs are captured as digests, a later evaluator can ask whether the same context and candidate set would produce the same decision, and can attribute any difference to a specific changed input rather than to unexplained model variance.

4.4 Semantic gates

A core application of System One is semantic routing, expressed through a routing structure that descends from a global context graph through topics, concepts, skills, and MCP capabilities or resources to a semantic gate, then to an SCW and finally an agent, runtime, or tool. A semantic gate is a location in the context graph where a transition can be evaluated, and it has two functions at once: it is a semantic location and it is an admission boundary.

System One may select the gate. The gate does not itself grant authority; the host evaluates the proposed transition at the gate. The sequence is therefore semantic selection → gate → admission, never semantic selection → immediate execution. This is what allows semantic routing to become more sophisticated without handing the decision model unrestricted authority over the tool layer.

System One may select the gate; the gate does not itself grant authority — the host admits the transition

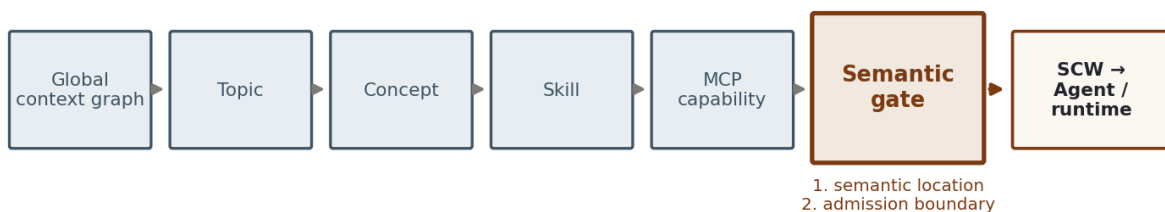


Figure 9. A semantic gate is a dual-function control point: a location in the context graph and an admission boundary. Selection and authorization remain separate operations.

4.5 Negative decisions and the error taxonomy

A robust decision system must not collapse every non-success into “deny.” System One distinguishes at least six outcomes, because they carry different meanings and demand different responses. Conflating them is one of the most common reasons agent failures are hard to

diagnose.

NoDecision	the decision system produced no candidate at all
Decision(Deny)	the decision layer explicitly returned a negative decision, where that decision type supports denial
ProviderUnavailable	the decision engine could not be queried
MalformedDecision	a decision was returned but failed schema or semantic validation
AdmissionDenied	a valid decision existed, but the host refused execution
ExecutionFailure	an admitted operation failed at runtime

Separating these states makes the system substantially easier to evaluate: a routing benchmark can isolate NoDecision and MalformedDecision, a security benchmark can isolate AdmissionDenied, and a reliability benchmark can isolate ProviderUnavailable and ExecutionFailure, none of them contaminating the others.

5 Applications of the decision layer

System One should be evaluated not as a single application but as a platform capable of hosting many decision experiments over the same address space. The applications below share one structure: System One answers “which candidate?” and the host answers “may this candidate be used?” Each is a concrete instance of decision $\neq$ authorization.

5.1 Semantic routing

Traditional routing uses explicit rules — if the task type is finance, route to the finance agent. System One permits a structured alternative: given the current SCW, task semantics, candidate SCWs, candidate capabilities, policy constraints, state, and provenance, it returns a typed decision such as “selected SCW-Finance, decision_type = context_route, reason = semantic_match, required_authority = financial-analysis.” The admission layer then decides whether that transition is allowed. Crucially this makes routing observable and testable as two separate measurements: did System One select the semantically appropriate context, and did the admission layer correctly prevent unauthorized transitions? A routing failure and a security failure are no longer the same event.

5.2 MCP routing

MCP introduces a routing surface of its own. An agent may have access to many MCP servers, each exposing many tools and resources under different authentication and capability scopes. The semantic problem — which capability is relevant to this task? — is distinct from the security problem — is this capability authorized for this task? System One addresses the first; the host answers the second. An MCP gate becomes an explicit transition point recording source SCW, destination service, requested capability, semantic reason, authority, policy, decision, admission, execution, and result. This turns external tool access into an observable semantic boundary rather than a direct exposure of every available tool to the model.

5.3 A2A delegation

A2A defines independent agents, tasks, messages, artifacts, and context identifiers for grouping related interactions, and is designed so that independent agents can communicate without accessing one another's internal state, memory, or tools. System One can act as a delegation-selection layer: given a task, context, candidate agents, capabilities, and constraints, it selects a candidate agent, and the host then evaluates whether delegation is permitted, what information may cross the boundary, what capability the receiving agent obtains, and what provenance accompanies the handoff. The architecture therefore treats an A2A handoff as a controlled transition between address spaces rather than merely another model message.

5.4 Cross-SCW bridges and cross-window communication

Communication between worlds occurs through explicit bridges rather than implicit inheritance. A bridge may define direction, message type, allowed fields, a transformation, authority, expiration, provenance, and destination, so that the destination receives a permitted representation rather than the complete source context. System One can propose the receiver; admission decides whether the transition is allowed. Because the bridge is explicit, cross-context data transfer becomes an observable, minimizable event, and a research platform can measure directly whether explicit bridges reduce accidental context transfer relative to an architecture in which any reachable state may flow.

5.5 Context retrieval and memory partitioning

System One also applies to context selection. Rather than presenting an agent with an undifferentiated pool of possibly relevant information, the system constructs a bounded candidate set and selects the best-fitting member under the task, context, and constraints. The essential restriction is that candidate generation and admission remain separate: a semantically relevant document does not automatically become admissible context. Three states — retrieved, admitted, and used — must not be conflated. Agent memory raises the same problem across working, episodic, and durable regions: without explicit partitioning a model may receive information simply because it was technically retrievable. Treating memory regions as resources associated with a context lets System One decide which region is relevant, admission decide which region may be accessed, the runtime record what was actually retrieved, and observation record which records entered the execution context — a measurable boundary around context expansion.

5.6 Agent, loop, and maker/checker/judge selection

Maxey0 describes “how” in terms of loops, agents, skills, and execution mechanisms, so System One can also select among candidate execution patterns — a direct agent, a maker/checker pair, a planner/executor, or a retrieval, coding, or research loop. It determines which candidate best matches the supplied context; the host determines whether the selected loop is available and permitted. The maker/checker/judge pattern is a special case: System One can treat the roles as candidate destinations and decide whether checking is necessary, which checker is appropriate, whether a judge should be invoked, and whether a result requires escalation — while never becoming the final authority over execution. “Checker B is the appropriate candidate” does not imply “execute Checker B”; the host still evaluates availability, authorization, and policy.

5.7 Drift detection

A decision produced under context C_1 may later be executed under context C_2 . A naive system reuses the earlier decision because the two contexts look similar. Maxey0 instead makes decision validity depend on an explicit compatibility relation: a prior decision is reusable only if C_1 and C_2 are equivalent under a system-defined relation and the relevant policy, provider, lifecycle, and execution constraints remain compatible. This decomposes “drift” into semantic drift (the selected context changes for materially similar requests), policy drift (the same decision yields different admission results as policy changes), execution drift (the runtime produces materially different outcomes for equivalent admitted operations), and observation drift (equivalent executions produce different or incomplete provenance). When drift is detected, System One recommends a remediation path — recompute the decision, move to a new SCW, invalidate the cache, or request human approval — and the host enforces it.

5.8 Decision caching and cache poisoning

A semantic decision can be expensive, requiring model inference, retrieval, candidate generation, tool discovery, and context construction, so caching is valuable. But a cache is not an authorization, and a cached decision is not an admission grant. A decision cache key must therefore include enough state to establish whether the earlier decision still applies — a digest over request, context, state, candidate set, provider, model, version, policy, SCW, and contract.

$$\text{Valid} = \text{Freshness} \wedge \text{SemanticCompatibility} \wedge \text{PolicyCompatibility} \wedge \text{ContextCompatibility} \wedge \text{ProviderCompatibility}$$

A cache hit means only that a previously computed decision may be reusable; the admission controller must still evaluate the present state. Caches also introduce an attack surface, since an adversary may try to make an incorrect decision reusable. System One therefore proposes a validation sequence — provider decision → schema validation → semantic validation → provenance binding → cache admission — and retains, with each cache record, the originating context and candidate digests, provider, model and policy versions, SCW, contract, creation and expiration times, provenance, and decision identifier, so an entry can be invalidated precisely when its originating conditions no longer hold.

5.9 Application taxonomy

The applications above, together with several more, form a common family that the platform can evaluate under one harness. The taxonomy is worth stating explicitly because it defines the surface across which the same decision / admission / execution / observation contract is exercised.

1	Semantic location selection	the appropriate topic, concept, skill, gate, or SCW
2	Context admission candidate selection	which contextual region should receive a task
3	MCP routing	among tools, resources, and servers
4	Agent selection	the appropriate agent from a bounded candidate set
5	Loop selection	the execution pattern best suited to the task
6	A2A delegation	a remote or local agent for delegated work
7	Cross-SCW bridge selection	an appropriate transition between contextual worlds

8	Context retrieval	relevant contextual state from bounded candidates
9	Maker/checker/judge routing	verification and adjudication paths
10	Drift remediation	a corrective path when context or state changes
11	Decision caching	whether an existing decision remains applicable
12	Replay	reconstruct and rerun historical decisions
13	Resource-aware routing	among candidates under latency, cost, or resource limits
14	Multi-agent topology planning	an agent graph or delegation sequence
15	Policy explanation	structured, non-authoritative rationale for a selection

Two of these deserve a note. Resource-aware routing lets System One fold latency, cost, token consumption, model quality, network access, human-approval requirements, execution time, and memory into the semantic selection, creating an opportunity to study whether explicit address-space semantics help agents make better resource decisions without directly controlling execution authority. Policy explanation is deliberately marked non-authoritative: the system may produce structured metadata describing the semantic basis of a selection, but that explanation is never the record on which authorization depends.

6 Provenance, observability, and replay

6.1 Provenance

Every transition should retain provenance, forming a causal chain from context through decision and location to admission and execution. A trace links run, task, decision, location, SCW, contract, and execution identifiers, and each step carries provider, model and version, input and context digests, the candidate set, the selected destination, the policy version, the admission result, the tool or MCP server or A2A task involved, timestamps, errors, artifact digests, and cache status. The goal is not logging but reconstructability: an evaluator should be able to answer what context existed when the decision was made, which candidates were available, which was selected, which policy evaluated that selection, which SCW contained the execution, which tool or agent was actually invoked, and what changed afterward.

6.2 Observability

Maxey0 extends conventional execution telemetry by making context location an explicit part of every trace. A minimal observation record binds run, task, decision, admission, and execution identifiers to source and destination SCW, contract, provider, model and version, candidate and context digests, policy version, admission result, tool, server, timestamp, and result digest. That single record supports analysis at several levels — model (what did the model decide?), context (what information was available?), routing (which candidate was selected?), policy (was the transition admissible?), execution (what actually occurred?), and outcome (what state resulted?). These levels are what allow a failure to be localized rather than attributed generically to “the AI.”

Observation is a research object, not a debugging afterthought

A system cannot meaningfully evaluate execution-space control if it cannot determine where the agent was, what it saw, what it selected, what it was allowed to do, and what it actually did. Observable execution is therefore an architectural requirement, and the platform must preserve enough state to reconstruct the causal chain without relying on the model to remember it. A model can forget, a context window can be compacted, a session can be resumed, and a cached decision can outlive the invocation that created it — so system provenance needs an external representation independent of the model's internal state.

6.3 Determinism, replay, context versioning, and lifecycle

Explicit decision objects make replay possible. A replay system reconstructs the original request, context, candidate set, model and provider, model and policy versions, SCW, decision, admission state, and execution environment, and then asks whether the replayed decision equals the original. Any difference can be attributed to a specific cause — model change, provider change, context change, candidate change, policy change, environment change, or nondeterminism — which is far more informative than comparing final natural-language outputs.

Replay depends on versioning. A decision is bound to the context version against which it was produced: $\text{Decision}(D, C_{v1})$ does not automatically imply validity under C_{v2} , even when the two contexts are nearly identical by similarity, because semantic similarity is not compatibility. The system should determine explicitly whether a change affects authorization, candidate set, task identity, state, capabilities, policy, or execution environment. Lifecycle enters the same way. SCWs and their regions can be ephemeral, session-scoped, durable, or immutable/versioned, and lifecycle becomes part of decision validity: a decision that references ephemeral state should not automatically survive the state that generated it. The architecture also distinguishes durable or reference context from working or scratchpad state, so that private working state is intentionally difficult to bridge into another world without explicit transformation — a property a research platform can test for by asking whether selective context propagation is safer than treating all agent state as transferable.

7 Security model, threat model, and invariants

7.1 Identity and authority

An agent can have identity without authority, and it can be assigned a capability without every semantically relevant action becoming authorized. Recent standards work identifies agent identity, authorization, auditing, non-repudiation, and prompt-injection mitigation as areas requiring attention as agents receive access to diverse data and applications. System One operates on identity and capability metadata without becoming the authority that grants them, giving the sequence identity → candidate selection → authorization → execution rather than identity → model decision → automatic authority.

7.2 Excessive agency

Frameworks for LLM risk identify excessive agency — excessive functionality, permissions, or autonomy, with model outputs allowed to cause downstream actions — as a major hazard, triggered by hallucination, prompt injection, compromised extensions, or compromised peer

agents. System One is designed to reduce the architectural coupling between a model decision and action authority. The model determines that a candidate appears to satisfy the supplied semantic requirements; the authorization layer determines that the candidate is permitted under the current identity, policy, capability, and execution state; the runtime determines that the operation was actually executed. This yields three independently testable failure classes — decision failure, admission failure, execution failure — and the architecture never assumes they share a cause.

7.3 Threat model

The research platform models both model-level and system-level failures explicitly. Stating them as a catalog is what allows each to be reproduced, injected, and measured rather than discussed anecdotally.

Prompt injection	malicious content attempts to alter agent behavior directly
Indirect prompt injection	external documents, sites, tool outputs, or retrieved content influence later execution
Cross-SCW leakage	information from one SCW becomes visible to another with no explicit bridge
Capability leakage	an agent gains tools or resources outside its intended capability set
Shared-environment leakage	agents share files, credentials, network, or durable state
Routing manipulation	an attacker causes System One to select an unintended destination
Confused deputy	a lower-trust component induces a higher-authority one to act on its behalf
Stale decisions	a decision is reused after its contextual assumptions changed
Cache poisoning	a malicious or incorrect decision is inserted into the cache and reused
Provider failure	the model produces malformed, inconsistent, unavailable, or manipulated output
Unauthorized delegation	an agent delegates without appropriate authority or information controls
Provenance loss	the system cannot determine which context, decision, policy, or agent acted
Policy drift	the policy governing a decision changes after the decision was created
Hidden state	execution depends on state that was not represented in the decision record

These divide naturally into two families. The first — injection, routing manipulation, confused deputy, provider failure — attacks the decision itself. The second — leakage, stale decisions, poisoning, provenance loss, hidden state — attacks the space around the decision. The architecture’s central bet is that separating decision from admission and binding both to provenance addresses the second family structurally, even when the first family succeeds in manipulating the model.

7.4 Security invariants

The platform enforces several explicit invariants. Each names a class of transition and supplies a predicate a monitor can evaluate, so that a violation is a discrete event rather than a judgment call. These invariants matter more to the platform than any individual model prompt, because they can be tested without assuming the model is reliable.

1	Decision $\neq$ Authorization	a selection is never, by itself, permission to act
2	Authorization $\neq$ Execution	permission to act is not the act; the runtime is separate
3	CachedDecision $\neq$ AdmissionGrant	a cache hit is reuse of a decision, not a grant of authority
4	SemanticSimilarity $\neq$ Authorization	relevance never implies permission
5	Addressability $\neq$ Admissibility	being reachable as a candidate is not being allowed
	NoExplicitBridge $\Rightarrow$	
6	NoImplicitCrossSCWTransfer	absent a bridge, state does not cross a world boundary
7	Admission $\Rightarrow$ CurrentPolicyEvaluation	every admission is evaluated against present policy and state
8	Execution $\Rightarrow$ ObservableExecutionRecord	nothing executes without leaving a reconstructable record
	DecisionReuse $\Rightarrow$ Context $\wedge$ Policy $\wedge$ State	
9	compatibility	a decision is reused only when its originating conditions still hold

7.5 The security model

System One is not a complete security boundary by itself. The architecture instead defines boundaries at which controls can be applied, under one primary assumption: the model is not the final authority, and the enforcement system must remain able to reject a System One decision. The security architecture therefore contains at least three independent functions — semantic selection, authority enforcement, and runtime isolation — performed respectively by the model, the host policy engine, and the sandbox or execution runtime. Observation adds a fourth function that does not prevent an action but makes it reconstructable. No single layer is assumed to provide all four properties, and a bad semantic decision should merely produce a bad candidate proposal; a security failure occurs only if that proposal can execute despite lacking authority.

8 Relationship to existing agent infrastructure

8.1 System One and current agent architectures

The public shape of contemporary agent platforms makes the System One research problem concrete. A managed agent API can expose an agent harness that handles sessions, orchestration, context compaction, and recovery, while the application supplies tools and chooses the execution environment, which may be hosted or external. Such platforms explicitly distinguish an agent session, an SDK session, a conversation, and a sandbox as different resources. That is a useful external reference: agent architecture already contains multiple distinct resource boundaries. Maxey0 proposes making one further boundary explicit — the semantic execution-space boundary — without claiming that any provider currently implements SCWs or System One. The mapping is a research mapping, not an implementation claim.

8.2 Codex-style safety and execution-space control

Published coding-agent safety work describes clear technical boundaries, human approval for higher-risk actions, and telemetry sufficient to understand and audit agent behavior, along with operating-system restrictions, reduced permissions, process-tree propagation of constraints, network controls, and file-write restrictions. These controls address physical execution

authority. Maxey0's question is complementary: can semantic execution-space identity be represented and enforced alongside physical execution boundaries? A physical boundary such as "Sandbox-17" establishes where code can execute; a semantic boundary such as "SCW-17" establishes why that execution belongs there, with a decision that selected it, a policy that admitted it, and a runtime that executed it — a richer causal chain than a sandbox identifier alone.

Human approval remains an independent authority mechanism throughout. System One may identify that an action requires approval, but the classification is not the authorization: it routes an external write to a high-risk path, admission requires human approval, the human approves or rejects, and the runtime executes only if approved. This is directly relevant to coding agents, where approving nearly every command destroys productivity while approving nothing is unsafe — a tradeoff addressed in Section 10.

8.3 Relationship to standards

Maxey0 is complementary to existing standards rather than a replacement for them. MCP provides tool and resource interoperability; Maxey0 can determine where those capabilities belong within a contextual execution graph. A2A provides agent-to-agent communication; Maxey0 can represent the contextual and address-space relationship governing it. IAM provides identity and authorization; Maxey0 can provide semantic context and decision provenance around the authorization event. Sandboxes provide physical execution isolation; SCWs provide semantic execution-space identity. Agent harnesses coordinate loops, tools, contexts, and execution; Maxey0 provides an explicit contextual address-space abstraction across those mechanisms. This composition is the primary research proposition.

8.4 The complete conceptual stack

Assembled, the layers form a stack in which a human or application issues a task, the Context Plane locates it, System One selects a candidate, admission authorizes the transition, the Execution Plane performs it, and the observation layer records it. The same six services can be one process in a small deployment or a distributed system in a large one; the architectural contract, not the deployment topology, is what matters.

Six logically separable services, one architectural contract, many deployment topologies

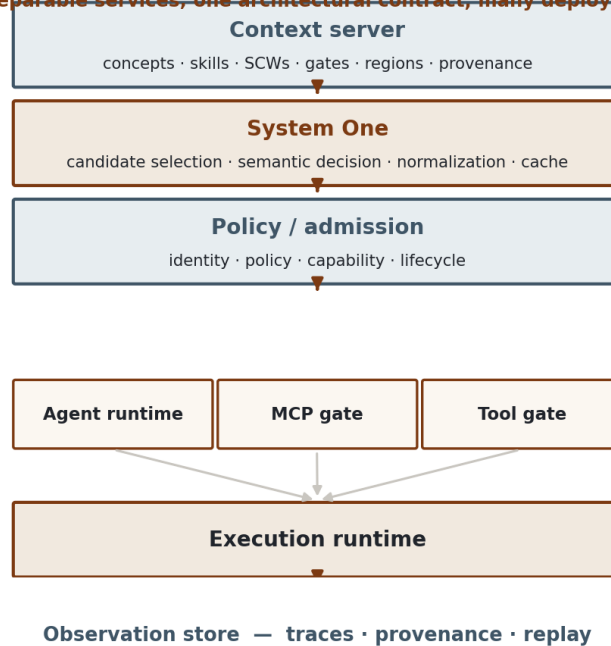


Figure 10. Six logically separable services realize the architecture: context server, System One, policy/admission, the gated runtimes, the execution runtime, and the observation store.

9 The Maxey0 control architecture: SCW0 and the Dyson interpretation

Beneath the individual worlds sits a foundational control world, SCW0, whose purpose is to provide the bounded environment from which other SCWs, agents, skills, loops, and relationships are governed. Its principle is that a multi-agent system should have an observable, assignable, controllable, and enforceable address space — stronger than merely placing agents inside containers. The system should know which agent belongs to which context, which context belongs to which task, which resources are addressable, which transitions are permitted, which policy controls them, and which execution resulted.

Maxey0 uses a Dyson-inspired conceptual model to describe increasingly structured autonomous systems. The analogy is about the organization of intent and meaning, not energy collection. A swarm is a distributed collection of agents and SCWs surrounding a model; a bubble adds mechanisms for detecting semantic drift and maintaining relationships among components; a shell is a strongly bounded execution architecture in which SCW0 establishes mapped paths, constraints, and resource relationships around the system. System One operates inside this structure as the mechanism that selects among structured possibilities.

The Dyson interpretation: organization of intent, not energy

SCW0 · mapped paths, constraints, resource relationships

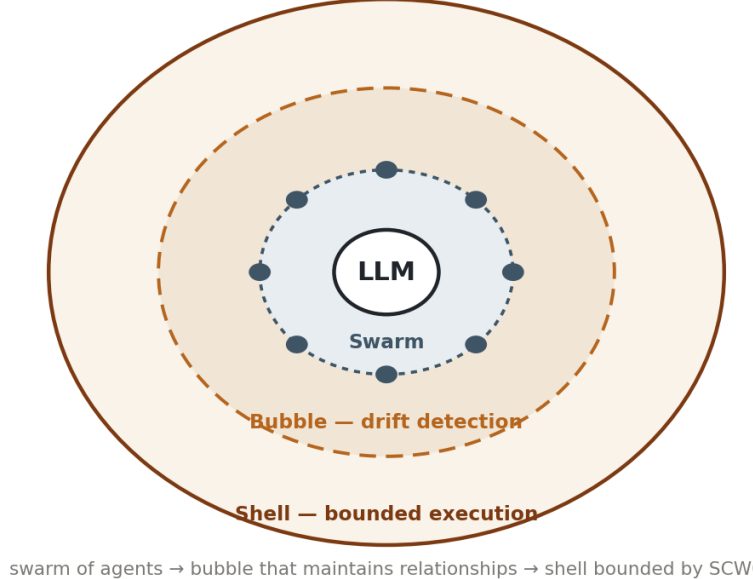


Figure 11. The Dyson interpretation: swarm, bubble, and shell describe how intent is organized around a model, with SCW0 defining the outer bounded execution architecture.

A foundational principle closes the control architecture: an agent should not be the ultimate authority over the boundaries in which it operates. An agent may propose “I should move to SCW-B,” but it should not thereby establish “SCW-B is now my authorized execution space.” The host retains authority over address-space creation and admission, giving a clean control-plane separation: the agent proposes, System One selects, SCW0 governs, admission authorizes, the runtime executes, and observation records. The intent is to reduce the possibility that an agent can redefine its own execution boundary through reasoning alone.

10 Evaluation methodology

The architecture should be evaluated empirically rather than accepted as a conceptual improvement, and the evaluation must compare systems with and without each component. A stable platform interface — decision request → candidate provider → decision engine → native decision → admission controller → runtime → observation store — lets the decision engine change while the surrounding architecture stays fixed, so that different models, prompts, deterministic algorithms, embeddings, symbolic selectors, and hybrid selectors can be compared under one contract. The recurring question is whether an improvement comes from SCW semantics or merely from one model routing better.

10.1 Experimental conditions

A central requirement is a no-SCW baseline, so that the contribution of each layer can be isolated. The condition matrix adds one architectural element at a time.

A - - - - baseline: conventional agent, no explicit worlds

B	yes	-	-	-	isolated SCW contribution
C	-	yes	-	-	isolated decision-layer contribution
D	yes	yes	-	-	combined architecture
E	yes	yes	yes	-	decision-cache behavior
F	yes	yes	yes	yes	drift and long-lived execution
G	yes	yes	yes	yes	adversarial evaluation

10.2 Proposed experiments and workloads

Six experiments follow from the conditions. Semantic routing constructs tasks with multiple plausible SCWs and compares direct model routing, rule-based routing, and System One routing on correct destination selection, ambiguity, latency, and downstream success. Unauthorized-transition prevention builds worlds with conflicting authority boundaries and attempts transitions that are appropriate-and-authorized, appropriate-but-unauthorized, inappropriate-but-authorized, and both inappropriate and unauthorized, measuring whether admission correctly separates suitability from authority. Prompt-injection resistance introduces untrusted instructions into documents, tool outputs, MCP responses, shared memory, and delegated-agent messages, and distinguishes decision, admission, execution, and observation manipulation. Context-leakage tests place controlled secrets in a source world and attempt transfer through inheritance, retrieval, tools, MCP, shared files, and explicit bridges. Decision-cache-validity runs repeated requests while changing policy, identity, candidate availability, context, and permissions, measuring execution under invalid decision state. Provenance-reconstruction gives investigators only the logs of a long multi-agent workflow and measures whether they can reconstruct who acted, where, under what decision and authority, with what execution and result.

These map onto workload classes that stress different properties: independent research tasks (cross-task leakage, routing, contamination), shared-artifact tasks (legitimate collaboration versus unauthorized state transfer and conflicting modification), tool routing (correct selection versus unauthorized or unnecessary tool use), delegation (destination selection, information transferred, unauthorized delegation, attribution), drift (stale acceptance, detection, invalidation, recovery), and adversarial context (routing manipulation, cross-boundary leakage, capability escalation, admission bypass). A separate metric governs candidate generation, since System One cannot select an object that was never generated: candidate recall must be measured apart from decision accuracy, and a routing failure attributed to the correct destination never being generated is a different failure from one where it was generated but not selected.

10.3 Research hypotheses

The following are proposed hypotheses, not established findings. Each is falsifiable, and the architecture would be weakened if a no-SCW baseline matched it on the relevant metric.

H1 Boundary attribution	explicit SCW identity increases the share of transitions for which source and destination execution spaces can be identified
H2 Context isolation	explicit SCW boundaries reduce unintended cross-context information transfer versus an equivalent no-SCW system
H3 Capability attribution	explicit semantic gates increase the ability to attribute tool/capability selection to a particular decision
H4 Decision/authorization separation	separating decision from admission reduces cases where model output directly determines execution authority
H5 Provenance completeness	binding decisions, admissions, and executions to identifiers increases reconstructability of agent behavior
H6 Cache efficiency	decision caching reduces repeated computation without materially raising stale-decision acceptance when checks are enforced
H7 Drift recovery	explicit context and policy identity improves detection of decisions that became invalid after state changes
H8 Provider neutrality	the same SCW and admission architecture supports multiple decision engines without changing core safety properties

10.4 Metrics

The platform measures both safety and productivity, because a system that prevents every action by denying everything is not a useful autonomous-agent architecture. The objective is not maximal restriction but controlled execution.

Routing	top-1 accuracy, top-k recall, incorrect-destination rate, routing consistency
Isolation	cross-SCW leakage, unauthorized state transfer, capability leakage, unauthorized bridge creation
Authorization	unauthorized-execution rate, admission false positives / negatives, policy-bypass rate
Provenance	trace completeness, boundary-attribution rate, decision-to-execution linkage, provenance-loss rate
Cache	hit rate, latency and token reduction, stale-decision acceptance, invalidation accuracy, poisoning detection
Drift	detection latency, false-positive / false-negative rate, recovery time
Productivity	task completion, time to completion, tool and model calls, token consumption, human approvals, retries

A complete evaluation also includes ablations — removing SCW identity, semantic scope, policy binding, provenance, admission, decision caching, explicit bridges, runtime binding, or observation one at a time — so that the architecture is not credited with a property produced by only one component.

10.5 The safety-productivity tradeoff

The desired relationship can be written as $\text{Utility} = \text{Productivity} - \text{Risk} - \text{Control cost}$, where control cost includes human approvals, latency, model calls, policy evaluations, context construction, and routing computation. System One adds computation; the research must determine whether that computation buys enough control benefit to justify its cost. One promising application uses semantic decisions to classify when an operation is routine versus

when it warrants additional review — routing a local file read to an existing authorization while routing an external production mutation to human approval — without letting the model decide that an operation is authorized. The measurable question is whether contextual routing reduces unnecessary approvals while preserving the required safety boundary.

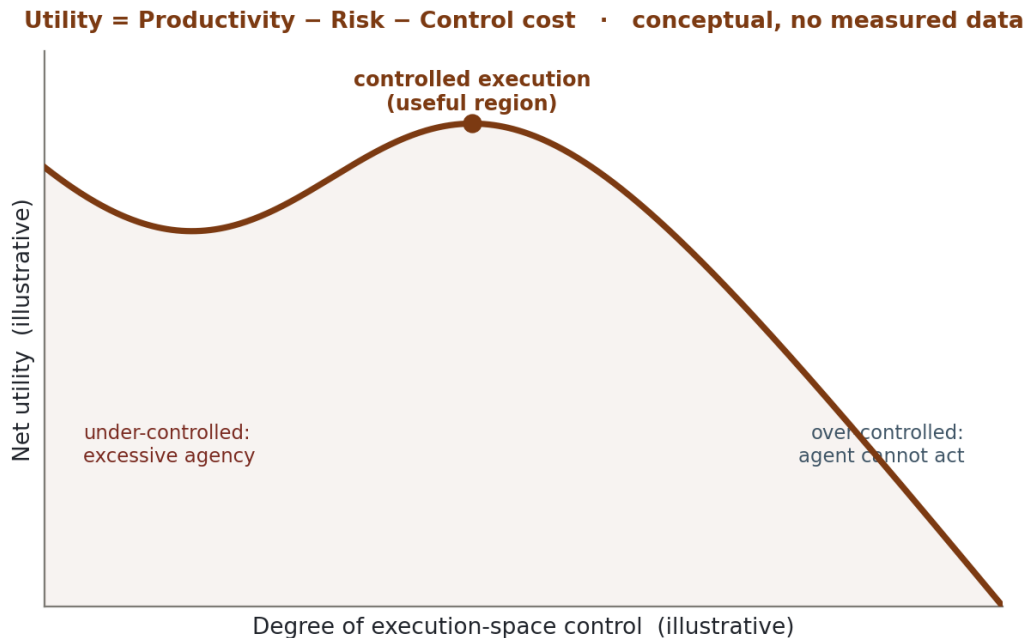


Figure 12. The tradeoff the architecture targets. Utility is expected to peak at intermediate control, not at unrestricted agency and not at total restriction. The curve is conceptual and carries no measured data.

11 Implementation

11.1 Implementation architecture

A practical implementation organizes into six logically separable services — a context server (concepts, skills, SCWs, gates, regions, provenance), System One (candidate selection, semantic decision, normalization, cache), policy/admission (identity, policy, capability, lifecycle), the gated runtimes (agent runtime, MCP gate, tool gate), the execution runtime, and the observation store. They need not be separate network services; a small deployment may combine them and a large one may distribute them. A corresponding source layout separates context, decision, admission, execution, observation, and evaluation packages, and this is an architectural proposal rather than a claim about the current state of any particular repository.

11.2 MCP gates as control points

An MCP gate is a concrete implementation point for the architecture. The gate receives the source SCW, requested capability, target service, decision identifier, admission state, policy reference, and provenance, and evaluates whether the request is consistent with the current admission; if authorized it forwards the request, and if not it rejects the transition. The gate is therefore more than a network proxy — it is an execution boundary at which semantic intent, authority, and external capability intersect, producing the enforceable path SCW → System One → admission → MCP gate → MCP server → external resource.

11.3 A staged research program

The architecture can be built and evaluated incrementally, so that each layer is measured before the next is added.

1	Formalization	SCW, context-graph, gate, decision, admission, and provenance schemas
2	Baseline	a no-SCW agent system; routing, leakage, tool selection, latency, cost, provenance
3	SCW	explicit boundaries without System One; measure the change
4	System One	provider-neutral decision vs deterministic and model-based routing
5	Admission	separate decision from authority; test policy violations and confused-deputy cases
6	Multi-agent	A2A or equivalent delegation; measure cross-agent state propagation
7	MCP	multiple servers and capability classes; capability routing and authorization
8	Cache	decision caching; efficiency versus stale acceptance
9	Drift	context and policy mutation; invalidation and recovery
10	Adversarial	injection, capability-leakage, routing-manipulation, shared-environment attacks
11	Reproducibility	replay and provenance evaluation
12	External comparison	against existing agent-harness capabilities

11.4 Reference implementation requirements

A reference implementation should expose SCW creation and destruction, SCW addressing, SCW state versioning, policy binding, context admission, decision requests, decision provenance, cross-SCW bridges, tool and MCP gates, execution binding, observation, replay, decision caching, cache invalidation, and drift detection. It should preserve enough information to answer what happened, why it was selected, which context requested it, which policy permitted it, what authority was used, what actually executed, what information crossed the boundary, and what state resulted. Those questions define the minimum observability requirement, and a reproducible experiment additionally records code revision, provider and model versions, prompt and skill versions, SCW and policy definitions, candidate set, context and state digests, decision and admission results, execution environment, tool versions, timestamps, randomization settings, and artifact digests.

12 Limitations and what System One does not claim

The proposal has substantial limitations, and stating them is part of the method. Semantic scope is difficult to define precisely, and a poorly designed SCW taxonomy may produce unnecessary fragmentation. Semantic routing is itself susceptible to model error; System One does not eliminate incorrect reasoning. Admission systems can contain their own implementation errors, so separating decision from authorization does not guarantee correct authorization. Provenance systems can become complex and expensive, and excessive instrumentation can increase storage and latency. Explicit boundaries can create coordination costs, and a no-SCW baseline can outperform an SCW architecture on tasks where the additional structure is unnecessary.

Model providers may already implement overlapping capabilities, the architecture may be hard to standardize across heterogeneous frameworks, and a semantic boundary cannot substitute for physical isolation where physical isolation is required. The architecture also does not automatically prevent covert channels: two agents may communicate through timing, shared external resources, or model outputs unless those channels are separately controlled.

What System One is not

System One is not an authorization engine, an IAM replacement, an operating-system sandbox, a general autonomous agent, a replacement for an agent harness, MCP, or A2A, a guarantee of model correctness, a guarantee against prompt injection, a guarantee of safe execution, a claim that semantic similarity is authority, or a mechanism that should grant itself capabilities. This negative definition is architecturally load-bearing: the value of System One depends partly on what authority it does not possess. A bad semantic decision should merely produce a bad candidate; a security failure occurs only when that candidate can execute without authority.

Maxey0 likewise is not simply a prompt-management system, a vector database, an agent router, a sandbox manager, or an observability layer. It connects these concerns through an explicit contextual address-space abstraction whose central question is where a computation is allowed to exist, what context belongs there, and how the system controls and observes movement between execution spaces.

The architecture should be explicitly falsifiable. It would be weakened if experiments show that existing harnesses already provide equivalent semantic address-space guarantees, that explicit SCW identity produces no measurable isolation benefit, that explicit gates do not improve attribution, that provenance records do not improve reproducibility, that System One adds latency and complexity without measurable benefit, that decision caching provides no useful efficiency, that SCW boundaries do not reduce cross-context leakage, or that no-SCW systems perform equivalently on the relevant safety and productivity metrics. Architectural complexity is not, by itself, evidence of architectural value.

13 Discussion and conclusion

The increasing availability of autonomous-agent infrastructure makes the distinction between semantic decision and execution authority increasingly relevant. Current systems already contain many of the required primitives: managed orchestration, subagents, MCP support, and sandbox-backed execution; explicit execution boundaries, approvals, network controls, and telemetry; standards work on identity, authorization, auditing, non-repudiation, and prompt-injection controls; interoperable task and context mechanisms among independent agents. These developments indicate that the problem space is becoming a systems problem rather than solely a model-quality problem. Maxey0's proposed contribution is to make execution-space identity and semantic addressability explicit within that systems architecture.

Stated as a formal transition model, the system state at time t is $X_t = (G_t, P_t, I_t, R_t, O_t)$ — the SCW graph, policy state, identity and authority state, runtime state, and observation state. A decision is $D_t = f(X_t, Q_t)$; admission is $A_t = g(D_t, X_t)$; execution is $E_t = h(A_t, R_t)$; and observation is $O_{t+1} = k(E_t, O_t)$. A secure transition requires that a non-empty execution implies admission was allowed, and that admission implies the decision fell within the permitted decision domain — so authorization never silently authorizes an arbitrary operation merely because a related

semantic decision was permitted.

Execution occurs $\Rightarrow A_t = \text{allow}$ · $A_t = \text{allow} \Rightarrow D_t \in \text{permitted decision domain}$

The strongest version of the research claim is therefore deliberately narrow. It is not that Maxey0 solves agent safety. It is that Maxey0 provides a testable architectural abstraction for studying whether explicit semantic execution spaces improve the control, attribution, isolation, provenance, and evaluation of autonomous agents — and that System One provides the decision mechanism through which that abstraction is exercised without becoming the authority over execution. Autonomous agents should not be treated as operating in an unbounded context; their execution should be associated with an explicit, addressable, observable, and enforceable contextual space.

The principal research object, in one line

Address → Decide → Admit → Execute → Observe. Semantic model reasoning stays useful without becoming the final authority over execution, and the whole architecture can be evaluated with and without SCWs, with and without System One, with and without caching and drift detection, and across different models and frameworks. The next step is empirical: implement it, compare it against meaningful baselines, and let the measurements of routing, isolation, leakage, authorization, provenance, replay, cache validity, drift recovery, latency, cost, approval burden, and task productivity decide whether the abstraction earns its place.

Appendix A Core formal model

Context graph: $G_C = (V_C, E_C)$. Candidate generation: $CANDIDATES = C_G(x, \Gamma)$. Decision: $d = D(x, CANDIDATES, \Gamma, S, P, M, V)$. Admission: $a = A(d, I, \Gamma, S, P, K)$. Execution: $e = E(a, R)$. Observation: $o = O(x, d, a, e)$.

**$x \rightarrow CANDIDATES \rightarrow d \rightarrow a \rightarrow e \rightarrow o$ with $d \neq a$, $a \neq e$, $Cache(d) \neq a$,
 $SemanticSimilarity \neq Authorization$**

Appendix B Decision, admission, and execution contracts

The three records are intentionally separate; together they provide the causal chain Decision $\rightarrow$ Admission $\rightarrow$ Execution, each with its own identifier.

```
Decision { decision_id, request_id, run_id, task_id, source_scw_id, destination_candidate_id,
decision_type, selected_candidate, candidate_set_digest, context_digest, state_digest,
provider, model, model_version, policy_version, contract_id, provenance, timestamp,
cache_status, expiration }
```

```
Admission { admission_id, decision_id, identity, policy, capability_state, result, timestamp }
```

```
Execution { execution_id, admission_id, runtime, environment, tool, input_digest,
output_digest, result, timestamp }
```

Note on sources and claims

This paper describes the Maxey0 research architecture and proposes System One as a research application of the Maxey0-SuperSpace model. The authors do not represent the proposed architecture as an established industry standard or a demonstrated production safety system. Where the paper discusses external agent infrastructure, the statements are based on publicly available documentation and publications available as of September 2026. Where it describes Maxey0 concepts — SCWs, System One, SCW0, semantic gates, decision caching, and related architecture — those descriptions represent the research program and proposed experimental architecture unless an external implementation or result is explicitly identified. No experimental result is claimed unless supported by a documented experiment.

References

- [1] OpenAI. Introducing the Agents API. September 10, 2026.
- [2] OpenAI. Agents API. OpenAI API documentation. Accessed September 2026.
- [3] OpenAI. Multi-agent. OpenAI API documentation. Accessed September 2026.
- [4] OpenAI. Sandbox Agents. OpenAI API documentation. Accessed September 2026.
- [5] OpenAI. Running Codex safely at OpenAI. May 8, 2026.
- [6] Wiesen, D. Building a safe, effective sandbox to enable Codex on Windows. OpenAI, May 13, 2026.
- [7] National Institute of Standards and Technology. Accelerating the Adoption of Software and Artificial Intelligence Agent Identity and Authorization. Initial Public Draft, February 5, 2026.
- [8] National Institute of Standards and Technology. New Concept Paper on Identity and Authority of Software Agents. February 5, 2026.
- [9] OWASP GenAI Security Project. LLM06:2025 — Excessive Agency.
- [10] Agent2Agent Protocol. A2A Protocol Specification, Version 1.0.0. Accessed September 2026.
- [11] Model Context Protocol. Specification and documentation. Accessed September 2026.

- [12] Maxey0. Maxey0-SuperSpace: Structured Context Worlds as Enforceable Address Spaces — an introductory research paper on context isolation, semantic addressing, authority boundaries, and observable AI execution. Maxey0 research manuscript (foundational project material).